Machine verification and certification of research mathematics

Executing what we claim

Jeppe Rich

DTU Management

August 14, 2026

What this talk covers

1. The idea
2. The tools
3. Numerical and symbolic verification
4. An example from our own work
5. A paper from the literature
6. The workflow
7. Limitations, and why this is the easy version
8. What it takes to install and run
9. Conclusion

Everything shown here has been run. The reports, the scripts and the raw output are on disk and re-runnable.

1. The idea

Three things proofreading does not catch

1. A dropped term in a long algebraic rearrangement. The equation still looks like an equation.
2. A calibration that does not reproduce its own table. The parameters are stated, the percentages are stated, and no one substitutes one into the other.
3. A heuristic reported as an optimum. The number is plausible, the solver returned it, and nothing in the paper distinguishes "best found" from "proven best".

All three survive any number of readings. All three are caught immediately by execution.

The asymmetry we exploit

Producing a result is expensive. Checking a produced result is cheap.

Claim	Producing it	Checking it
An algebraic identity	a careful hour	`simplify(L-R)==0`
A Taylor expansion	by hand, error-prone	`series(expr,x,0,n)`
A calibrated parameter	a fitting exercise	substitute and compare
An integer optimum	NP-hard search	compare bound to incumbent
A transport plan	an LP solve	recompute marginals and cost

Verification rides on the gap between the two columns. It is not a second research project.

The discipline, in one rule

Every claim in the paper is either **executed** or **explicitly listed as not executed**.

There is no third category. No claim gets to be "obviously fine".

This is what makes the output an audit rather than a test suite. A test suite tells you what passed. An audit tells you what passed, what failed, and precisely what was never looked at.

The boundary of the check is part of the deliverable. A report that hides its own coverage is worse than no report, because it buys confidence it has not earned.

Two halves of the same idea

The mathematics

The equations in the manuscript.

Translate each to SymPy and run it.

Verdicts: **PASS** / **FAIL** / **NOT-CHECKABLE**

`/verify-math`

The computed results

The numbers the code produced.

Hand each to an independent oracle.

Verdicts: **CERTIFIED** / **CHECKED** / **FAIL** / **UNVALIDATED**

`/verify-model`

The equations and the code are separate failure surfaces. A paper can have correct algebra and a buggy solver, or a clean solver and an inverted formula. Both halves need checking.

2. The tools

The stack

Layer	What it is	What it gives us
SymPy 1.14	computer algebra system, Python	the judge for every symbolic and numeric claim
jr_optlib	our shared optimisation library: 40 vetted primitives, 12 oracle modules, 143 oracle-backed tests	the judge for every computed result
Oracle bank	OR-Library instances with published optima (`scp41`, optimum 429)	known answers to compare against
Registry	`functions.yaml`, `instances.yaml`, `references.yaml`	"does a trusted version of this already exist?"
Two skills	`/verify-math`, `/verify-model`	the automation that wires a paper to the judges

What the model does, and what it does not do

The language model does exactly one job: translation. It reads the manuscript, finds the claims, and writes them as executable expressions.

It never renders a verdict. SymPy and the oracles do that.

Why this matters

A model that reasons about whether an equation is correct can be confidently wrong. A model that writes `simplify(LHS - RHS)` and runs it cannot: either the residual is zero or it is not.

Every report prints the exact expression that was checked next to every verdict. A wrong translation is therefore visible in the report rather than hidden behind a green **PASS**.

What you get back

Two artifacts per run, in a timestamped folder inside the project:

- `math_check_report.md` or `coverage_map.md`
  The audit. One row per claim, keyed to your own equation labels, with the verdict, the expression checked, and got-versus-stated.
- `verify.py` or `verify_model.py`
  The re-runnable script. Self-contained. It goes into the reproduction package and a referee can execute it.

The report is not a summary of the run. It is the run, transcribed.

3. Numerical and symbolic verification

Symbolic: closing the identity

Every labelled equation is classified into one bucket, and each bucket has one method.

Bucket	Method	Passes when
`IDENTITY`	`simplify(expand(L-R))`	the residual is exactly 0
`SERIES`	`series(expr,x,x0,n)`	the coefficients match the stated ones
`SOLVE`	`solve(Eq(...),var)`	the solution set matches the closed form
`MOMENT`	`sympy.stats`	the moment matches the stated expression

When a sum has a symbolic upper limit that will not close in general, the script instantiates it at $J \in \{2, 3, 4\}$ and passes only if all three give zero. The report states which sizes were used, so the reader knows a finite check stood in for a general one.

Numerical: the claims nobody re-substitutes

Every explicit number in the paper is recomputed from its stated inputs: table cells, worked examples, calibrated parameters, quantiles.

Rounding is respected. A computed 0.2609 matches a stated 0.26 and fails a stated 0.24.

This is where papers actually break

Of the failures we have found in our own manuscripts, the large majority are numeric, not algebraic. The algebra gets checked by a co-author. The number in the third column of Table 4 does not.

Oracles: verify, never re-solve

For computed results there is no "LHS minus RHS". Instead we ask what property the true answer must have, and test for it. Ordered by strength:

1. feasibility and objective recomputation, independent of the producing code
2. duality or relaxation certificate: if $\mathrm{UB} = \mathrm{LB}$, optimality is proven
3. exact-versus-heuristic differential on the same instance
4. planted or seeded instances with a known answer
5. small-instance exhaustive enumeration
6. benchmark libraries and trusted reference implementations
7. monotonicity and metamorphic invariants

Levels 2, 4, 5 and 6 *certify*. The others *check*. The distinction is the whole point of the next slide.

Four verdicts, and why there are four

CERTIFIED	A certifying oracle passed. Optimality or uniqueness is proven for this instance.
CHECKED	Feasible, objective recomputed, consistent with every applicable check. Not proven optimal.
FAIL	An oracle rejected the result. This is a provable defect, not a warning.
UNVALIDATED	No applicable oracle exists. Reported as such, never quietly passed.

Collapsing **CERTIFIED** and **CHECKED** into "verified" would be the single most tempting and most dishonest simplification available. A heuristic that lands on the optimum without a bound is **CHECKED**, and the paper must say so.

4. An example from our own work

`Pub_OptimismBias_PartA`: 58 checks in one run

Outcome	Count	What it covered
PASS	55	structural identities, the Taylor series, the moment inversion, every lognormal moment, the full-sample and by-sector tables
FAIL	3	one empirical calibration, three inconsistent numbers
NOT-CHECKABLE	12	CLT and SLLN steps, definitions, asymptotic conditions

The twelve **NOT-CHECKABLE** rows each carry a one-line reason. That list is what tells a referee, and us, exactly where the machine stopped and human judgement takes over.

The three failures were one real problem

The manuscript states an empirical calibration $(\hat{\mu}, \hat{\sigma})$ and, in the same passage, the median and mean overruns it implies.

Claim	Stated	Computed	Check
early median, $e^{\hat{\mu}} - 1$, $\hat{\mu} = 0.14$	$+18\%$	$+15.0\%$	**FAIL**
late median, $e^{\hat{\mu}} - 1$, $\hat{\mu} = 0.04$	-1%	$+4.1\%$	**FAIL**, sign flip
late mean, $e^{\hat{\mu}+\hat{\sigma}^2/2} - 1$	$+4\%$	$+7.9\%$	**FAIL**

A median of -1% requires $\hat{\mu} = \ln 0.99 = -0.01$, not $+0.04$. The reported pair fits neither the median nor the mean.

The comparable Flyvbjerg calibration in the same paper is internally consistent and passed. The check found the one place it was not.

And on the code side: a coverage map

`Pub_MIPEntropy_MPC`, verified against `jr_optlib` oracles:

```
CERTIFIED  IPF 2D relaxation, 40x40
           marginal residual 2.27e-13; scaling form 2.98e-14 [certifies]
CERTIFIED  exact min-cost transport LP, 30x30
           obj 187.77 vs scipy 187.77, rel diff 1.5e-16 [certifies]
CERTIFIED  SCP scp41 / Gurobi-30s:  obj 429 vs Beasley opt 429
CHECKED    SCP scp41 / RICH-1-4:  obj 432 vs Beasley opt 429
           feasible, cost recomputed, bounded below by the optimum
```

The heuristic row is **CHECKED**, not **CERTIFIED**. It is a good solution, it is provably not optimal, and the map says exactly that.

Six projects run so far

Project	Check	Claims	Outcome
`Pub_OptimismBias_PartA`	math	58	55 pass, 3 fail, 12 **NOT-CHECKABLE**
`Pub_NapstiGranularity`	math	107	85 pass, 11 fail, 14 **NOT-CHECKABLE**
`Pub_SAA_PMIP_MC`	math		run
`Pub_PopInt_Part2`	both		run
`Pub_MIPEntropy_MPC`	model		coverage map above
`Pub_PopInt_PartB`	model		plus three targeted probes

The two runs with counts above total 165 checked claims and 14 failures. Every one of those failures was a real inconsistency in a manuscript we had already read many times.

5. A paper from the literature

Kahneman and Tversky (1979)

Prospect Theory: An Analysis of Decision under Risk, *Econometrica* 47(2), 263–292.

Why this paper:

- It is one of the most cited papers in economics. If anything has been checked, this has.
- Its argument is a chain of stated implications, not a computation. That is the hard case for a machine check, and therefore the honest test.
- Its mathematics is elementary. Nothing hides behind machinery.

We transcribed every derivation in the paper into `KT1979_claims.tex` with page numbers, verbatim, adding nothing and repairing nothing, and ran it.

How you check an implication, not an equation

The paper never says "$A = B$". It says "preference X implies inequality Y".

Write each inequality as a residual the paper asserts is positive. A valid one-step derivation from premise residual P to conclusion residual C is exactly the existence of a positive multiplier:

$$C \;=\; m \cdot P \quad \text{identically,} \qquad m > 0 \;\; \text{under the paper's own assumptions.}$$

SymPy is asked to close `simplify(C - m*P) == 0`. The multiplier is printed for every check, so the reader sees exactly which quantity was assumed positive.

Concavity of v is encoded as the chord inequality it actually is, $v(\lambda a) - \lambda v(a) \geq 0$, and fed in as a premise like any other.

A worked line

Problem 1, page 266. The modal preference $B \succ A$ under expected utility gives

$$u(2400) > 0.33\,u(2500) + 0.66\,u(2400),$$

and the paper states this implies

$$0.34\,u(2400) > 0.33\,u(2500).$$

```
eq:allais-eu | IMPLICATION | PASS | C - m*P = 0  [m = 1]
```

Problem 3: $u(3000) > 0.8\,u(4000)$ implies $u(3000)/u(4000) > 4/5$.

```
eq:p3-ratio   | IMPLICATION | PASS | C - m*P = 0  [m = 1/u4000]
```

The multiplier $1/u(4000)$ is the division the paper performs silently. Making it explicit is what turns prose into something executable.

Result

Checks run	**49**
PASS	**49**
FAIL	0
FLAG (unstated side condition)	**1**
Strictness gaps noted	3
NOT-CHECKABLE (recorded, not run)	6

Every stated implication in the paper follows from its stated premise. The algebra of prospect theory is sound, forty-seven years on.

The six **NOT-CHECKABLE** rows: the response data, the editing-phase definitions, the Appendix representation theorem, the endpoint behaviour of π, Figures 3 and 4, and one assertion the paper makes without giving a derivation to check.

The one thing it found

Page 285, the proof that regular insurance is preferred to probabilistic insurance. The paper bounds

$$V \;=\; \pi(p/2)f(x) + \pi(p/2)f(y) + \big[\,1 - 2\pi(p/2)\,\big]\, f(y/2)$$

and then writes: *"using the concavity of f, it suffices to show"* the same expression with $f(y/2)$ replaced by $f(y)/2$.

SymPy computes the discarded gap exactly:

$$\text{gap} \;=\; \big(f(y) - 2f(y/2)\big) \cdot \big(\pi(p/2) - \tfrac{1}{2}\big).$$

Concavity forces the first factor to be ≤ 0. The gap therefore has the sign the argument needs only if $\pi(p/2) \leq 1/2$, which the paper never states. If $\pi(p/2) > 1/2$, the "it suffices to show" step runs the wrong way.

What that finding is, and what it is not

It is not an error. The condition is mild, and the paper's own Figure 4 puts π below the diagonal over most of the range.

It is a step where a substitution is made without recording the condition that licenses it. That is exactly the class of thing a human reader glides over and a machine cannot.

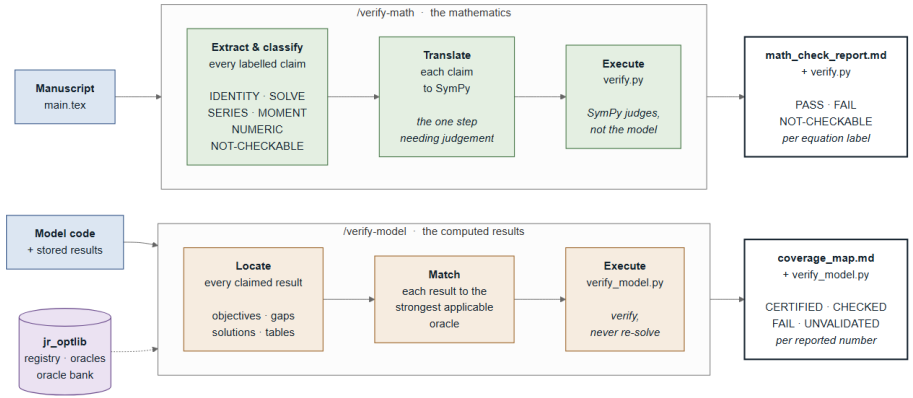
Three further rows assert a *strict* inequality from a concavity argument that yields only $\geq$. Strictness needs strict concavity, which the paper assumes informally but does not invoke at those points.

The point

A famous, heavily scrutinised, mathematically elementary paper still has four places where the stated hypotheses do not quite carry the stated conclusions. Our own manuscripts will not do better unaided.

6. The workflow

The workflow



Only the **translation** step needs judgement. Everything downstream of it is execution.

A **FAIL** sends you back to the manuscript. An **UNVALIDATED** sends you to `jr_optlib` to add the missing oracle.

7. Limitations, and why this is the easy version

This is not formal verification

A proof assistant (Lean, Coq, Isabelle) checks a proof against axioms. Every inference step is machine-checked, and the trusted base is a small kernel.

We do none of that. We check that stated computations are the computations claimed.

Three honest limits

1. **The translation is trusted.** If the LaTeX is formalised wrongly, the verdict is about the wrong claim. Mitigation: the report prints every expression checked.
2. **Instantiation is not generality.** Passing at $J \in \{2, 3, 4\}$ is strong evidence, not a proof for all J.
3. **Reasoning is out of scope.** Limit theorems, existence, uniqueness and convergence arguments land in **NOT-CHECKABLE**. They always will.

Why the easy version is the one worth having

	Proof assistant	What we do
Guarantee	the theorem is true	the stated computations are right
Effort	months per paper, a specialist	minutes per paper, the author
Prerequisite	the whole theory formalised	a `.tex` with labelled equations
Coverage	total, within what is formalised	partial, and it says which part
Failure mode	you cannot finish	you get a report with gaps
Adoption	essentially zero in our field	runs today, on every paper

Full formalisation is stronger and, for applied transport and economics papers, not going to happen. A check that runs is worth more than a proof that does not.

Where the errors actually are

The errors that reach print in applied work are overwhelmingly computational, not logical: a dropped term, an inverted formula, a parameter that does not reproduce its table, a heuristic reported as an optimum.

Those are exactly the errors this catches, and exactly the errors a proof assistant would spend months getting into position to catch.

The trade we are making

We give up the guarantee that the theorem is true. We keep the guarantee that the paper says what the mathematics says, and we get it cheaply enough to run on everything.

8. What it takes to install and run

What you need

For	You need
the mathematics	Python with SymPy. That is the whole dependency.
the results	Python with NumPy and SciPy, plus `jr_optlib`. Solver extras (PuLP, Gurobi, NetworkX, POT, numba) only if your problem uses them.
the automation	Claude Code, with the two skills in `~/.claude/skills/`.
your paper	a `.tex` file with `\label{}` on the equations. The labels are what the report keys to.

No LaTeX toolchain is required for the check itself. No proof assistant. No new file format.

A paper with unlabelled equations still works; the report anchors to a short quote instead. Labelling them is a five-minute investment that makes the report readable.

Install and run

Once, per machine

```
python -m pip install sympy numpy scipy pyyaml
git clone https://github.com/jepperich69/jr_optlib
cd jr_optlib && pip install -e .[test]
pytest                                          # 143 oracle-backed tests
```

Per paper

```
/verify-math  -project Pub_MyPaper
/verify-model -project Pub_MyPaper
```

Both run as background agents and open the report when done. Add `--inline` for a small file you want checked immediately.

Output lands in `Pub_MyPaper\_math_checks\<timestamp>\` next to the manuscript, not in a temporary directory.

What to write in the paper

The wording matters, because the temptation to overclaim is real.

Say this

"Checked against independent oracles: feasibility and objective recomputation, duality certificates, differential tests against exact solvers, and metamorphic invariants (`jr_optlib@<commit>`)."

For a specific number: "the reported optimum was certified by a zero duality gap."

Never say this

"Formally verified." "Proven correct." "Machine-checked proof."

Pin the library commit at submission and freeze it. A shared library that keeps improving and a paper that must stay reproducible coexist only if the paper names a SHA.

9. Conclusion

Conclusion

1. **Checking is cheaper than deriving.** That asymmetry is what makes routine verification affordable, and it is the only reason this works.
2. **The model translates; the machine judges.** No verdict is ever an opinion.
3. **Coverage is part of the output.** NOT-CHECKABLE and UNVALIDATED are first-class verdicts. A report that hides its own boundary is worth less than none.
4. **It finds real things.** Three inconsistent calibrations in our own manuscript. One unstated side condition in Kahneman and Tversky.
5. **It is not formal verification, and does not need to be.** It catches the errors that actually reach print, at a cost low enough to run on every paper.

Where to start

- Pick the paper you are least sure about. Run `/verify-math -project <name>`.
- Read the **NOT-CHECKABLE** list first. It tells you what you are still carrying alone.
- If a result has no applicable oracle, that is a gap in `jr_optlib`, not a dead end. Adding the oracle once serves every later paper.
- Ship `verify.py` with the submission. A referee who can run your check is a referee you have already answered.

Reports, scripts and raw output for everything shown:
`AI_auto\literature\verification_demo\KT1979\Publikationer\<project>\_math_checks\` and `_model_checks\`